

张目远

Agent 开发 | RAG / Agent 系统 | 后端工程

中国 · 西安 电话 19829773360 邮箱 charlizezmy@qq.com



教育经历

西北工业大学 国家“双一流”建设高校 | 工业和信息化部直属 | 原 985 / 211 工程 | 软件工程 · 硕士 (软件学院)

2024.09 - 至今

学位授予方: 西北工业大学

- GPA 3.77 / 4.1, 专业排名 top 10%。
- 学校 Title 采用学校官网表述: 坐落于陕西西安, 以航空、航天、航海等领域人才培养和科学研究为特色的国家“双一流”建设高校, 隶属于工业和信息化部。

西北工业大学 国家“双一流”建设高校 | 工业和信息化部直属 | 原 985 / 211 工程 | 软件工程 · 工学学士 (软件学院)

2020.09 - 2024.06

学位授予方: 西北工业大学

- GPA 3.54 / 4.1, 专业排名 top 15%, 保研至本校软件学院。

实习 / 工作经历

大疆 DJI | 研发与AI使能部 · 绩效管理组 | Agent 平台 · Redis Streams · SSE · 绩效系统

2026.07 - 至今

绩效评估 Agent 平台 (rd-points) — 系统级共建者 | Agent 评估链路 with 结果可靠投递

背景:

- 绩效系统: 定时把飞书 Colla、云效、JIRA 的任务单子拉取到 Platform, 再提交给 Sphere 做复杂度-工时评估 (通过提示词调用各部门编写的 Skill), 得到 base_score; 任务完成后再次由 Sphere 对完成情况进行 Review, 最终计算综合得分; 另有员工覆盖率、部门健康状况、任务总池、贡献度排行、问题人员清单等看板数据。
- Platform (Spring Cloud, 业务编排中枢) + Sphere (Python FastAPI, Claude Agent 执行引擎) + FE (React) 三仓库协作: Platform 通过 HTTP 创建 Run, Sphere 执行 Agent 并回写结果, FE 经 Cookie 鉴权访问看板。

我的职责:

- 参与 Agent 评估链路共建: 理解 Platform 的 workflow 编排内核 (business_run 状态机、Transactional Outbox、ResultHandler 结果写回) 与 Sphere 的 Run 生命周期 (pending→running→终态、DB CAS 执行闸门、Redis Streams 五 lane 队列), 并参与结果可靠投递改造。
- 对 rd-points-platform 与 rd-points-sphere 两个仓库做深度盘点 (幂等/锁/Redis 三大维度穷尽搜索、biz 16 个子包逐类梳理、Confluence 详设 vs 代码实现核实), 沉淀为可复用的模块认知与问题清单。

技术关键词:

Spring Cloud / FastAPI / Agent / Skill / Base Score / Workflow 编排 / Redis Streams / Transactional Outbox / JIRA / 飞书 / 云效

Sphere 评估结果 Redis Stream 可靠投递改造 — 核心参与者 | webhook → Redis Streams 架构演进

- 背景：**
- Platform 发起异步评估后，Sphere 在 Run 终态时通过 HTTP webhook 回调 Platform；Platform 发布、重启或短暂不可用期间，有限次数回调重试可能耗尽，结果留在 Sphere 但未进入业务处理链，需要人工重投或重新评估。
- 指标与效果：**
- 设计目标：Platform 不可用期间完成的评估结果可持久保留，服务恢复后自动消费，无需人工重投；重复投递、消费者重启和发布过程不导致评估结果重复生效（at-least-once 投递 + effectively-once 业务生效）；成功/失败/取消三类终态可追踪、可重试、可告警。
- 我的职责：**
- 参与结果送达层改造：Sphere 侧 Run 终态与 sphere_result_delivery_outbox 同事务提交，Publisher 以 XADD 写入 Redis Stream（失败指数退避重试，超阈值进 DEAD_LETTER 并保留人工重放入口）；Platform 侧以 XREADGROUP + XAUTOCLAIM 认领 PEL 消息，仅事务提交成功后 XACK。
 - 参与灰度方案实现：result_delivery_mode 按 Run 灰度（WEBHOOK → REDIS_STREAM），灰度开关由 Apollo 配置中心热更新驱动，AtomicBoolean CAS 无锁切换，无需重启服务；保留 webhook 兼容路径与完整回滚步骤。
 - 参与幂等与故障场景设计：同一 business_run_uuid 终态重复消息仅记录并 ACK；同一 sphere_run_id 不重复创建有效 workflow_outbox 事件；XACK 前中断由 PEL 保留、重启后重新处理；确定性错误写 DLQ 不无限重试。
- 技术关键词：** Redis Streams / XREADGROUP / XAUTOCLAIM / PEL / XACK / Transactional Outbox / DLQ / Apollo 灰度开关 / at-least-once / effectively-once

贡献度助手 SSE 流式推送 — 核心参与者 | 长连接续传与 Redis 故障降级

- 背景：**
- 贡献度助手需要把 Agent 执行过程事件实时推送给浏览器；直接轮询存在空耗与延迟问题，需要长连接 + 断线续传 + 权限持续校验的完整链路。
- 我的职责：**
- 参与 SSE 长连接链路开发：GET /events 建立 EventSource 连接（最长 5 分钟），底层用 Redis Stream XREAD BLOCK 阻塞读取事件逐条转发，避免轮询空耗 CPU；连接到期后由浏览器原生自动重连，配合 cursor 做到用户无感续传。
 - 参与 cursor 续传与重置机制：Last-Event-ID 断线自动续传、after_event_id 手动指定起点；cursor 非法/超界/早于故障恢复点（recoveryFloor）时发送 stream.reset 全量快照重建前端状态。
 - 参与心跳双作用与降级路径：SSE 注释行保活 + 每 15 秒权限复审（权限被收回时服务端主动断开）；Redis 故障统一降级为 stream.degraded 事件通知前端，并用 Lua 脚本（Redis TIME）记录故障时间戳防止假连续。
 - 参与 Redis 原子性实现：Lua 脚本实现连接数限流（INCR + 超限回滚）、消息创建限流（分钟桶）、事件去重（SETNX 幂等）、fencing token 防旧执行轮次覆盖写入、MCP 委托 CAS 撤销。
- 技术关键词：** SSE / EventSource / SseEmitter / XREAD BLOCK / Redis Stream / cursor 续传 / fencing token / Lua 原子脚本 / 降级路径 / 权限复审

技术项目与沉淀

个人知识库 Agent | 端到端链路 Owner | 从 0→1 构建 Plan-Verify-Replan 工作流与三层 Memory 架构

Python + LangGraph + FastAPI；覆盖检索、验证、记忆、状态恢复与知识闭环

- 基于 LangGraph StateGraph 构建 Plan-Verify-Replan Agent，设计 Planner / Retriever / Answerer / Verifier / Replanner / KnowledgeLoop 六节点工作流；Verifier 基于 RAGAS Faithfulness 校验答案忠实性，未通过则触发重检索。
- 设计 Hybrid Search（text-embedding-3-small + ChromaDB + BM25）+ RRF 融合排序，引入 Query Expansion；构造 100+ 条 QA 评测集以 RAGAS 量化，Context Recall 相比 baseline 提升 36.6%，Faithfulness 达 0.843。
- 设计三层 Memory 架构：LangGraph InMemoryStore 管理会话级短期记忆，Redis Hash 持久化用户长期阅读偏好与兴趣标签，PostgreSQL 存储历史问答记录；Memory 命中时跳过重复检索，平均响应延迟从 602ms 降至 2ms。
- 基于 LangGraph Checkpoint (SqliteSaver) 实现 Step-level 状态持久化，检索或工具调用失败时自动回滚并触发降级策略（扩大 top-k / 放宽过滤条件），长链路任务故障恢复率 ≥ 95%。
- 构建知识闭环机制：Agent 回答时若检测到笔记覆盖不足，自动生成补充摘要写回知识库，持续扩充个人知识图谱。

基于 Spring Cloud 的电商交易系统 | 核心开发者

Spring Cloud 微服务架构；覆盖库存、订单、检索与缓存链路

- 基于 Redis + Lua 实现库存原子扣减，在秒杀压测场景下实现高并发请求的库存一致性控制与超卖防护。
- 引入 RocketMQ 构建异步订单链路，将下单流程异步化处理，实现削峰填谷并降低数据库瞬时写入压力。
- 采用 Redis + Lua + Redisson 构建分层并发控制机制，在库存与非库存场景分别实现原子性扣减与防重复下单控制。
- 采用 Canal + Binlog + MQ 构建最终一致性方案，以数据库变更为事件源，实现订单与库存的异步同步与状态修复。
- 基于 Elasticsearch 构建商品多维检索能力，支持复杂条件过滤与排序优化；设计 Caffeine + Redis 二级缓存，构建本地缓存优先的热点访问路径，压测环境下显著降低数据库访问延迟。

迭代框架重构与差分 Transformer 特征更新; 覆盖算法、训练与评测

- 针对 VideoSCI 重建中计算开销大、迭代后期特征更新停滞问题 (导致资源浪费与性能受限), 重构迭代框架, 解耦稳定结构与动态更新, 减少无效计算。
- 基于差分 Transformer 优化特征更新方式, 提升动态建模能力。
- 结果: PSNR 提升超过 0.5 dB, 参数量减少 50% 以上, 计算量减少 33% 以上, 实现更优的精度-效率权衡。
- 论文状态: AAAI 2027 投稿中 (用户确认)。

奖项与技能

Java / Python 后端开发: 面向对象设计与常用集合框架, 并发编程基础 (JUC、线程池、锁机制)

Spring Boot / Spring MVC / MyBatis / FastAPI, 理解 IoC、AOP 原理及常见设计模式

MySQL / Redis / MQ / Elasticsearch / ChromaDB 等中间件: 缓存设计、消息队列削峰、分布式一致性

大模型应用开发: RAG / Agent 系统完整落地经验; LangGraph / LangChain, Plan-Verify-Replan 工作流、多层 Memory 设计与 Checkpoint 失败恢复